Proyecto Final de Bootcamp – Guillermo Barrio

Esta es la memoria del Proyecto Final de Bootcamp (PFB) del bootcamp IA4 de Keepconding, realizado por Guillermo Barrio Colongues. Se trata de una ampliación de la práctica entregada del módulo LLM Engineering del mismo bootcamp. En esencia, el proyecto consiste en la creación de un dataset sintético a partir de uno que contiene datos electorales y socioeconómicos, todos públicos, para con él hacer el finetuning de un modelo preentrenado, un sistema de RAG evaluado por RAGAS, y un despliegue mediante Streamlit en una VM en Google Cloud. El seguimiento de parte de los procesos se lleva a cabo mediante LangSmith.

En el repositorio de GitHub se encuentran los siguientes directorios:

- **Directorio Notebooks**: Incluyen todos los usados en la práctica, que se mencionan en el resto de esta memoria, e incluyen el tratamiento inicial de los datasets electorales; la creación de los datasets sintéticos según distintos niveles; el proceso de fintuneado del modelo; la generación del RAG y su evaluación mediante RAGAS; y el código que genera la aplicación, incluyendo en ella un ejemplo trazabilidad en LangSmith.

- **Directorio Datasets:** Incluye una muestra de 500 filas del dataset original (datos electorales y socioeconómicos) con la columna de metadata; los datasets de train, test y validación utilizados en el finetuning; el directorio con los logs del proceso de finetuning; y las muestras de preguntas/respuestas del proceso de evaluación RAGAS.

- **Directorio App:** Incluye el código de la aplicación subido finalmente a Google Cloud.

- **Otros ficheros:** Esta memoria, la presentación del día 12, y el README.md.

En la **carpeta de mi Google Drive** en este enlace:

https://drive.google.com/drive/folders/1LwnIvE6j0IPL6HsE7xxsladEMRjIfMCI?usp=sharing

se encuentran todos los ficheros anteriores, además de otros que, por su tamaño, no hemos podido subir al repositorio, o por no ser realmente útiles:

- **Datasets:** los originales unificados de las 5 elecciones; así como el que incorpora los metadatos luego utilizados en la evaluación RAGAS. También está presente el directorio de las preguntas/respuestas asociadas al modelo que entró en overfitting.

- **Modelos:** El inicialmente finetuneado que resultó estar en overfitting, y el definitivo (FINAL_5_elecciones), ambos guardados en 4bits.

- **Ficheros de RAG de evaluación RAGAS**: Fichero .faiss y metadata.pkl.

- **Video de la demo de la app**: Demo_app.mp4. El vídeo se ha subido a YouTube en este link.

Dentro de mi **cuenta en Hugging Face**, se encuentran 3 modelos: el utilizado en la práctica de LLM Engineering que incluía solo las dos elecciones de 2019; el original ya con las 5 elecciones entre 2011 y 2019 que entró en overfitting; y el definitivo que hemos utilizado. Además, HF contiene el dataset unificado, con la columna de metadata vectorial.

Los nombres concretos de cada fichero se mencionan en el resto de la memoria.

Lecciones aprendidas y cosas a mejorar

En los capítulos siguientes detallaremos los pasos que hemos seguido y los resultados obtenidos, pero pienso que es necesario destacar las lecciones que he aprendido a la hora de hacer este PFB, al igual que en la práctica de LLM Engineering, y las cosas (muchas) que debo mejorar en el futuro, más allá de aspectos concretos del Proyecto. Se trata de una lista más o menos random, sin que sea necesariamente por orden de importancia.

- **Hacer el PFB en solitario ha podido ser un error**. No busqué necesariamente hacer el Proyecto en solitario, pero creo que hubiese habido lecciones y experiencias que tendría ahora en cuestión de hacer trabajo en equipo, que no tengo ahora. Quizás hubiese merecido la pena de apuntarme a otro equipo, o al menos intentarlo en serio al menos.

- **Importancia de un despliegue hecho pronto.** No me imaginaba lo difícil que iba a ser el despliegue del modelo en Google cloud, en especial poder conectar la VM con su GPU, y luego lo complicado de usar Unsloth en una maquina virtual. Mejor haber hecho un despliegue temprano, porque si es al final el sufrimiento sería exponencialmente mayor.

- **Vencer el miedo a la terminal.** Supongo que es algo que tendré siempre conmigo, pero es necesario el no tener miedo a la terminal, pese a los sinsabores que pueda dejar en procesos complicados como el despliegue de una aplicación en una máquina virtual.

- **Medir la ayuda de las LLM.** Tanto DeekSeek como Gemini han resultado fundamentales en el PFB, pero hay que vigilar lo que nos aconsejan. Es muy posible que el despliegue me haya resultado mucho más complicado de lo debido por dejarme aconsejar demasiado. Así mismo, el código que aconsejen puede estar sencillamente mal, o ser muy complicado de entender, y por lo tanto de debuggear.

- **Relato hecho temprano también.** Lo he intentado evitar, con éxito parcial, pero comenzar con el relato más bien pronto, sin dejarlo para el final, es muy importante.

- **El overfitting puede estar presente siempre.** Reconozco que me equivoqué con el primer finetuning de este PFB, ya que no supe reconocer los signos de ello en el propio finetuning, y después tampoco hasta al cabo de unos días. Lo describo en esta memoria, pero a veces algo 'muy bueno' significa 'muy malo'.

- **Uso aún insuficiente de plataformas de IA.** En este PFB he utilizado en ocasiones plataformas como LangChain o LangSmith, pero pienso que sigue siendo insuficiente. Es un paso que debo dar en el futuro, en especial de aquellas que tengan como especialidad el *retrieval,* que es lo que más ha fallado en este proyecto.

- **Más conocimiento de DevOps.** El despliegue de la app realmente lo hizo Gemini, porque, con excepciones también relevantes, en muchísimas ocasiones yo simplemente copiaba lo que el LLM me dictaba. Un mayor conocimiento, para empezar, de la consola de Linux, pasando por Google Cloud, me resulta imprescindible.

Introducción y planteamiento

Quizás lo más apropiado sea comenzar con las agendas que, cuando yo era pequeño, le regalaban a mi padre. En ellas, hacia el final, había 3 o 4 páginas dedicadas a los resultados electorales. Por aquel tiempo, principio de los años 80, no había habido demasiadas elecciones, pero siempre me quedaba prendido por sus resultados. Esta afición a las cifras de votos, porcentajes, participación y demás, siempre me ha interesado. La década pasada tuve una corta carrera política como asesor, donde una de mis funciones era precisamente hacer análisis electorales.

En 2020-21 cursé el bootcamp de Big Data y, como proyecto final, hicimos un análisis de resultados electorales de las elecciones generales en el conjunto de España entre 2011 y 2019. Los datasets, uno por elección en un total de cinco, incluían no solo las cifras de votos, porcentajes, etc.., sino también datos socioeconómicos como renta per cápita, afiliaciones a la Seguridad Social y otros. Existe un video en YouTube de una exposición del proyecto.

Los datos del dataset son todos públicos, y fueron extraídos de la web del Ministerio del Interior en el caso de las elecciones, y principalmente del INE para el resto. Los datos están desagregados a nivel de sección electoral, básicamente una o dos mesas electorales, de las que hay unas 36.000 en toda España. Ese es pues el número de filas de cada dataset, de nuevo, uno por cada elección general. El número de columnas rondaba las 90.

Posteriormente, de forma individual, hice el ejercicio de analizar los resultados de las elecciones autonómicas de Andalucía de 2022; en esta ocasión creé una aplicación mediante Plotly/Dash, y la alojé en Google Cloud.

En este bootcamp, como práctica del módulo de LLM Engineering, me preguntaba si sería posible entrenar (más bien '*finetunear*') un modelo pre entrenado para que fuera capaz de contestar de forma mínimamente inteligente a pregunta que le hagamos sobre la información que contengan los datasets. Quizás de forma demasiado prudente, tomé entonces solo dos elecciones generales, ambas celebradas en 2019, que fusioné, con lo que el dataset original contaba con algo más de 72.000 filas.

De esta forma, escogía un par de modelos pre entrenados (T5 y Gemma2) que, por separado, 'finetuneé'. Para ello tuve previamente que generar un dataset sintético supervisado de preguntas/respuestas. Tras el finetuning intenté, sin demasiado éxito, crear un RAG, que proporcione contexto al modelo. Y esa fue la práctica que presenté en el módulo LLM Engineering.

De cara al Proyecto fin de bootcamp, he ampliado la práctica, utilizando los 5 datasets de las elecciones, que hubo que optimizar, eliminado columnas que no aportaban realmente información útil, a la vez que hubo que hacer un curado de la columna del nombre del municipio, y crear otras como la que identifica el año y mes de la elección. Este proceso se llevó a cabo en el notebook **Unificador_Elecciones_11_19.ipynb**. Una vez unidos los 5 datasets, tenemos uno ya unificado y curado, llamado **elección_unificado.csv**, que cuenta con algo más de 181.000 filas y unas 76 columnas.

Capacidades	Práctica LLM Eng.	Trabajo Fin Bootcamp
Dataset de elecciones	2	5
Datasets curados	No	Sí
Dataset sintético	Sí	Sí
Finetuning Gemma2	Sí	Sí
Sistema RAG	No	Sí
Evaluación RAGAS	No	Sí
Tracing LangSmith	No	Sí
Despliegue GCP	No	Sí
Sistema memoria	No	Sí

En los capítulos siguientes iremos desgranando lo que hemos hecho en este PFB, pero conviene repasar las diferencias entre el PFB y la práctica de LLM Engineering, que se resumen en el cuadro anterior. Esencialmente, hemos añadido el RAG con su evaluación mediante RAGAS, así como el despliegue en Google Cloud utilizando una máquina virtual. La app tiene instalado un seguimiento de tracing con LangSmith y un sistema de memoria.

Los capítulos siguientes describen, por este orden:

- Creación del dataset sintético
- Finetuning del modelo
- Sistema de RAG
- Evaluación del RAG mediante RAGAS
- Despliegue en Google Cloud

Creación del dataset sintético

Ya desde la práctica del módulo LLM Engineering, era claro que iba a necesitar al principio crear un dataset con el que *finetunear* el modelo pre entrenado, otra cosa era cómo plantearlo. Por ejemplo, tenía que determinar qué tipos de pregunta, y cuantas de cada clase debería contener este dataset. Para ello, le pregunté a DeepSeek; no voy a aburrir al lector con las idas y venidas de mis conversaciones, pero al menos pienso que viene a cuento repasar el primer prompt que le hice:

> *Hola! Necesito que me aconsejes sobre el finetuning de un modelo de LLM pre entrenado relativamente sencillo, como por ejemplo sería el GPT-2. Estoy actualmente haciendo un bootcamp de Inteligencia Artificial, y concretamente el módulo de LLM Engineering. El trabajo a entregar de este módulo incluye precisamente el finetuning de un LLM pre entrenado.*
>
> *El finetuning consistiría en adiestrar a un modelo a contestar a preguntas sobre los datos de cinco datasets sobre resultados electorales en España, concretamente de cinco elecciones generales entre 2011 y 2019. Cada fila de los datasets representa el resultado en cada sección electoral, e incluye, además de los resultados electorales, diversos datos socioeconómicos. En total, cada dataset contiene unas 36.500 filas.*
>
> *Te adjunto una pequeña muestra de uno de los datasets, unas 40 filas, correspondiente a las elecciones generales de abril de 2019. Un par de ejemplos de preguntas a las que tendría que responder el LLM tras el fine-tuning serían estas:*
>
> *1) "Como ha evolucionado el porcentaje de voto al PP (columna '% PP') en las secciones electorales del municipio de Torrelavega entre las elecciones de 2016 y de abril 2019".*
>
> *2) "Como se correlacionan los porcentajes de voto al PSOE (columna '% PSOE') con la renta salarios de 2018 (columna 'Renta Salarios 2018') en el municipio de Burgos en las elecciones de abril 2019.*
>
> *Entiendo que para llevar a cabo el finetuning de un modelo pre entrenado es necesario un dataset con features y un target (supervised training), que lógicamente se compondría de cientos de preguntas como las dos que he mencionado, junto a sus respuestas. El problema es que no tengo ese dataset, por lo que habría que generarlo mediante el uso de un LLM potente, tales como tú, o GPT-5, por ejemplo. Entiendo también que el LLM preentrenado debería ser capaz de realizar operaciones matemáticas e incluir, si es posible, conocimientos básicos de distribuciones estadísticas.*
>
> *Necesito pues que me aconsejes cómo hacer el finetuning del modelo preentrenado, con las condiciones que te he descrito. En principio no es necesario que me proporciones código de Python, pero sí que me informes de aspecto como qué LLM pre entrenado sería apropiado tomar como base, y cómo generar el dataset de preguntas respuestas para el finetuning. Gracias!*

A grandes rasgos, DeepSeek me aconsejó en ese momento que partiese de un modelo T5-base, de 220 Mn de parámetros, por presentar un buen equilibrio entre tamaño y rendimiento. También mencionó el DestilGPT-2 y el Phi-2. Ya vimos en la práctica de LLM Engineering que esta elección fue un error, y que nos decidimos finalmente por un modelo Gemma2-9b, que es el único modelo que con el que hemos trabajado en el Proyecto Fin de Bootcamp.

Respecto al dataset sintético, DeepSeek en esencia me recomendó que lo generase no desde el principio con un LLM potente, como él mismo, debido al alto coste potencial sino que, partiendo de un total de preguntas/respuestas de unas 2.000, dividiese la creación en tres pasos o niveles:

- **Nivel 1** - Preguntas simples, básicamente búsqueda en el dataset: 60/70% del dataset;

- **Nivel 2** - Preguntas con algunos cálculos, como medias, encontrar el máximo, comparaciones: 20/25% del dataset; y

- **Nivel 3** - Preguntas elaboradas o complejas, estas sí utilizando un LLM: 10% del dataset.

Y eso fue esencialmente lo que hice tanto en la práctica de LLM Engineering como ahora. En esta ocasión partiendo de un dataset bastante mayor en filas, aunque algo menor en el número de columnas. Organicé la creación de cada nivel de peguntas en tres notebooks en Colab:

- **Generador_dataset_Nivel_1_PFB.ipynb** – En este notebook dividí la generación del dataset, a su vez, en tres partes: **1)** preguntas de búsqueda de un dato dentro de un determinado conjunto de columnas; **2)** búsqueda de dos datos; y **3)** búsqueda de un dato en cualquier columna. La pregunta y respuesta estaba definida por un texto tipo, del que en conjunto había unos 20 *templates*.

- **Generador_dataset_Nivel_2_PFB.ipynb –** En este notebook se generaron preguntas con algunos cálculos con 8 t*emplates* diferentes, que incluían comparaciones temporales, comparaciones geográficas, cálculos de diferencias entre partidos, y cuestiones similares.

- **Generador_dataset_Nivel_3_PFB.ipynb –** En este notebook se generan prompts que incluyen preguntas complejas, y sus respuestas que también lo son. He utilizado el modelo **v1 de DeepSeek**.

Finalmente se generaron 2.000 preguntas/respuestas del Nivel 1, 600 del Nivel 2 y 200 del Nivel 3. Tengo que reconocer que, sin la ayuda del código de DeepSeek, no hubiese podido generar los niveles 2 y 3, y en este último caso me ha resultado complicado especialmente el código. Las cosas como son. Con todo, el código original tenía algunos fallos, por ejemplo, cuando en alguna función debía hacer procesos repetitivos con ámbitos geográficos como sección, municipio, provincia, etc…, solo incluía uno de ellos, por lo que tuve que escribir los que faltaban. Tampoco nada grave.

Las preguntas y respuestas de los niveles 1 y 2 son relativamente simples, no así las del Nivel 3. En este último caso, por ejemplo, a la pregunta siguiente:

> *Considerando los datos electorales y socioeconómicos de los municipios de la provincia de Madrid (Fuenlabrada, Madrid y Parla) frente a los de Andalucía (Jaén y Baza), analice las diferencias en el comportamiento electoral, específicamente en el apoyo al PSOE y la participación, y relacione estos patrones con las variables de renta disponibles. ¿Qué factores contextuales podrían explicar estas divergencias geográficas y cuáles serían las implicaciones para una estrategia política nacional del PSOE?*

El modelo DeepSeek v1 empezó contestando:

> *El análisis comparativo revela diferencias significativas en el comportamiento electoral entre los municipios de Madrid y Andalucía. En términos cuantitativos, el apoyo al **PSOE** es notablemente más bajo en los municipios madrileños (Fuenlabrada: 30%,*

> *Madrid: 30%, Parla: 40%; promedio=33.3%) que en los andaluces (Jaén: 50%, Baza: 60%; promedio=55%). La participación electoral, sin embargo, es más alta en Madrid (promedio=66.7%) que en Andalucía (promedio=60%).*

Y unas 350 palabras después termina, por fin, de esta manera:

> *... La correlación observada sugiere que la mejora de las condiciones económicas en sus bastiones tradicionales podría, paradójicamente, erosionar su apoyo si no logra reconectar con un electorado cuyas prioridades y perfil socioeconómico evolucionan. La proyección indica que el PSOE enfrenta el desafío de ser simultáneamente un partido de gobierno arraigado y una fuerza competitiva en escenarios urbanos y plurales."*

Ahí es nada. Con perspectiva, quizás debí incluir algún tipo de limitación a la extensión de la respuesta, pero no deja de tener sentido. Respecto a los otros dos niveles, comprobé los datos en algunos casos, y eran correctos. Las respuestas estaban estructuradas en formato jsonl, que DeepSeek consideró era el más apropiado, con campos de 'instruction', 'input', 'output' y 'metadata'. El coste del uso del LLM en el nivel 3 fue muy pequeño, apenas 10 céntimos de euro.

Los datasets de cada nivel son los siguientes: **dataset_preguntas_nivel1_2011-19.jsonl**, **dataset_preguntas_nivel2_2011-19.jsonl**, y **dataset _nivel3_2011-19.jsonl**

El código también generó versiones de los datasets de los niveles 1 y en .json, que llamó 'pretty', así como back ups progresivos del dataset del nivel 3.

Naturalmente, ahora tenía tres datasets, pero, obviamente, debía unirlos de alguna manera para proceder al finetuning. Para ello le volví a preguntar a DeepSeek, que, esta vez, me dio un código que funcionó a la primera. Éste está alojado en el notebook **Unificador_Datasets_2011-19.ipynb**, e hizo básicamente tres cosas:

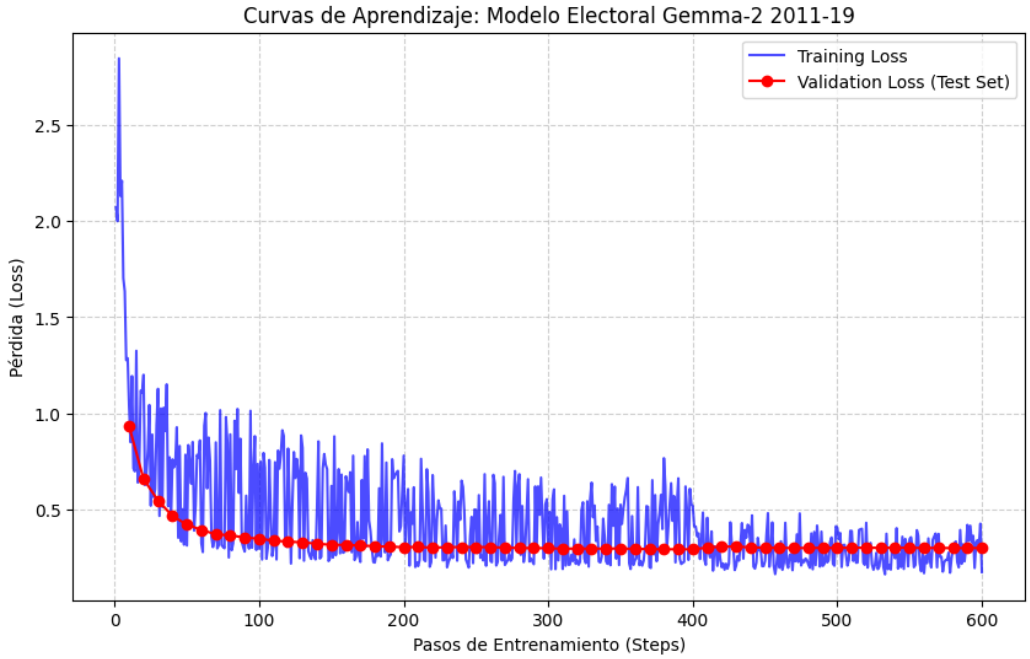
- **1)** unificar el formato de las preguntas respuestas, independientemente de su nivel;
- **2)** seleccionar aleatoriamente 2.000 preguntas respuestas, en la proporción 62% Nivel 1; 28% Nivel 2; y 10% Nivel 3; y
- **3)** crear los datasets de train, test y validación con pesos de 80, 10 y 10% respectivamente.

El resultado, es decir, los tres datasets de train, test y validación, junto a los .csv, que DeepSeek definió como formato HuggingFace, se hayan en el directorio Datasets repositorio de GitHub, en la carpeta **dataset_unificado_2011-19_corr**. Lo de '_corr' viene porque tuve que corregir los nombres de los municipios.

Proceso de finetuning

Tras tener el dataset sintético preparado, pasamos al fine-tuning del modelo Gemma2. En la práctica de LLM Engineering hicimos inicialmente el finetuning del modelo T5, pero resultó ser un fracaso. Probamos a continuación con el Gemma2-9b, y lo que obtuvimos fue algo más aceptable. Desde luego, de cara a este PFB nos hemos centrado únicamente en el este último modelo. Hemos utilizado la librería Unsloth, que comprobé que simplifica bastante el proceso de finetuneado.

El proceso de finetuneado se realizó en el notebook **Unsloth_Gemma-2-9b_2011-19_corr.ipynb.** En principio seguí el mismo proceso que utilicé en el caso de la práctica de LLM Engineering: utilizar 3 epochs, con un total de 600 batches. Las losses que fui obteniendo fueron muy parecidas a las que obtuve en la práctica de LLM, y fueron estas:

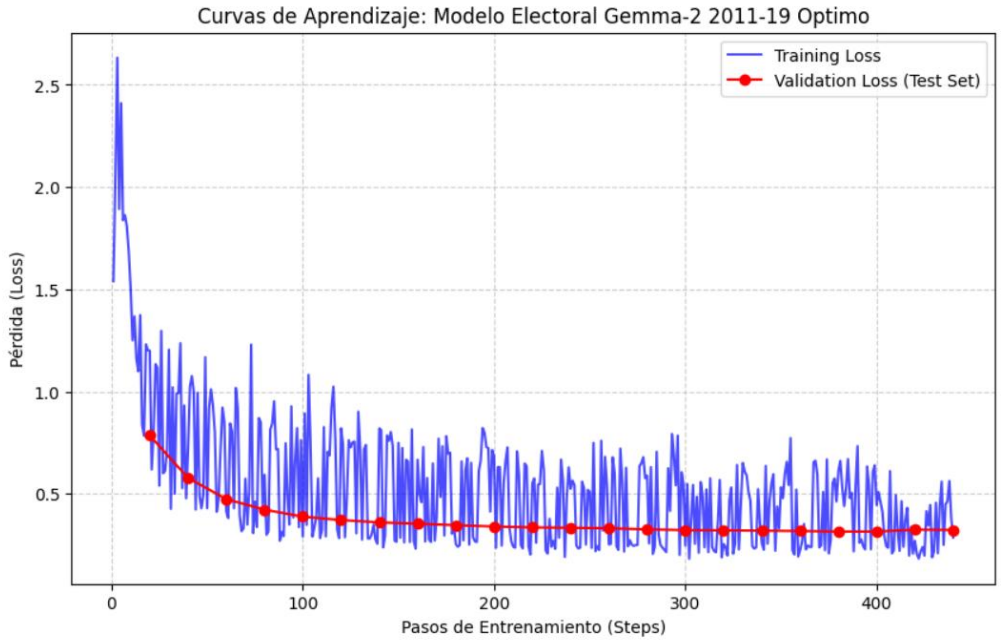


Vemos que el validation_loss es muy bueno, cercano al 0,30, lo que pasa es que este parámetro tocó un mínimo hacia el batch 400 y luego comenzó a subir ligeramente a lo largo de la tercera epoch. Vemos que allí, entre los batches 400 y 600 se produjo un descenso muy fuerte del training loss. Con perspectiva, me tuve que dar cuenta que ese es un signo de que el modelo está memorizando las respuestas, con lo que iba a sufrir de overfitting.

Desgraciadamente, ignoré este síntoma, y fue solo después de implementar el modelo que me di cuenta. Básicamente cuando, al hacerle una pregunta, contestaba tartamudeando, es decir, repitiendo un token constantemente, como 'el el el el el el …' y así durante filas y filas. Una forma de evitarlo podía ser el variando el parámetro del modelo que penaliza la repetición de un mismo token (*repetition_penalty*), e intentando retocar la temperatura, pero era claro, también para Gemini cuando le consulté, que el modelo estaba en overfitting, y que habría que hacer el finetuning de nuevo. De ahí el final del nombre del notebook '_corr'.

De todas formas, no borré el modelo en overfitting, una vez guardado en 4bits. Está presente en el Drive en la carpeta **'modelo-Gemma_2-9b_2011-19_finetuned-4bit'**, y el Hugging Face en este link **'GuillermoBarrio/gemma-2-9b-electoral-2011-19'**.

Para evitar el overfitting esta segunda vez, limitamos el proceso de finetuning a que el valiadation loss no aumentase más de dos batches seguidos, y lo cierto es que, en realidad, apenas necesitamos dos epochs. En la gráfica inferior, mostramos la evolución de los losses de train y validación, tocando esta última fondo en el entorno de 0.31, como cabía esperar. Vemos que ahora no tenemos los signos de que el modelo estuviese memorizando las respuestas.



Afortunadamente no hemos detectado que el modelo ahora 'tartamudee' como hacía el anterior, pese a poner una penalización por repetición inferior, en el entorno de 1,1. El modelo definitivo no está exento al 100% del 'tartamudeo', para ser completamente honesto, pero mucho menos frecuente. Este modelo está guardado en Drive en el directorio **'modelo-Gemma_2-9b_FINAL_5_elecciones'**, y en link **'GuillermoBarrio-Gemma_2-9b-electoral-5-elecciones'**. También hemos guardado los logs del finetuning en el directorio Dataset del repositorio de GitHub.

Proceso de RAG

La conveniencia de un sistema de RAG es clara: es necesario proporcionar un contexto adecuado al modelo ya 'finetuneado', ya que desde luego no se puede meter todo el dataset electoral en un prompt.

En la práctica del LLM Engineering intentamos sin éxito implementar un sistema de RAG. Primero le pedimos consejo a DeepSeek, que nos recomendó un complicadísimo código para reconocer el nombre de municipio y elección, que no llegó a ninguna parte. Algo de esperanza ya tuvimos con los consejos de Gemini, que indicó el uso de la librería FAISS.

Lo que hemos hecho con el RAG en la RFB se encuentra en notebook **'RAG_Gemma2_Practica_2011_19.ipynb'**. Se trata de llevar a cabo los siguientes pasos:

- Cargar el embedder de la librería transformers

- Preparar una columna en el dataset que reúna varios datos de cada fila

- Hacer el embedding de cada elemento de esta columna

- Crear el índice FAISS para agilizar las búsquedas

El segundo paso fue bastante complicado en la práctica, ya que hubo que añadir repetidas veces los nombres de los municipios y las elecciones, principalmente, para que FAISS pudiese reconocer las similitudes con la pregunta que le pasásemos. De esta forma, la columna, que llamamos 'metadata_vectorial', recogía para cada fila esta información, de la que mostramos un ejemplo:

> *'elecciones: Abril 2019. municipio: Abla. provincia: Almería. elecciones: Abril 2019. municipio: Abla. provincia: Almería. elecciones: Abril 2019. municipio: Abla. elecciones: Abril 2019. Resultados electorales de Abril 2019 en la sección 0400101001 del distrito 01 en el municipio de Abla de la provincia Almería. La participación electoral fue del 75.7%. El partido más votado fue el PSOE, el segundo fue el PP, el tercero fue el Cs, el cuarto fue el VOX, y el quinto fue el UP .Contexto socioeconómico: Renta media por persona de 9159.0 euros, con un porcentaje de afiliados a la Seguridad Social respecto a la población del 23.30%. Contexto demográfico: La sección tiene un porcentaje de mayores de 65 años del 27.0%, y un porcentaje de menores de 19 años del 14.0%. Perfil de voto: PP(19.53%), PSOE(42.73%), VOX(11.53%), Cs(17.17%), IU(0.00%), UP(5.77%).'*

También creamos otra columna llamada 'contexto', en la que almacenamos los mismos datos, pero sin la repetición inicial de elecciones, municipios y provincias, para utilizarla mejor en el contexto de la pregunta destinada al modelo, y no al FAISS.

El caso es que, al añadir y repetir estos datos, la selección de filas mejoró bastante, sin ser perfecta, pues la pregunta del usuario debía de tener, preferiblemente, un formato parecido a la información presente en la columna de metadata. Finalmente guardamos el fichero del indice .faiss (**electoral_index_2011-19.faiss**) y el metadata.pkl (**metadata_rows_2011-19.pkl**), que utilizamos posteriormente en el análisis de RAGAS. El dataset con las columnas de 'metadata_vectorial' y 'contexto' lo guardamos en mi Drive como '**eleccion_unificado_2011-19_metadata.csv**' y está subido también a HuggingFace como **GuillermoBarrio/dataset-electoral.** En el directorio Datasets del repo de GitHub hemos guardado una muestra aleatoria de 500 filas de este dataset, **eleccion_unificado_2011-19_metadata_sample_500.csv.**

De cara a la selección de filas para el RAG, hemos introducido un reranking, utilizando el modelo CrossEncoder de Transformers ‘*'cross-encoder/ms-marco-MiniLM-L-6-v2'*, que toma la selección del índice de FAISS, e incorpora una selección con más sentido semántico,

En el notebook aportamos un ejemplo de utilización del sistema de RAG con el modelo, en este caso preguntando por el municipio de Bargas (Toledo), y la respuesta parece correcta. Con todo, hay que recalcar que la pregunta textual fue esta:

> *¿Elecciones: Abril 2019: Cómo se relaciona la participación electoral con la renta personal en el municipio de Bargas en la provincia de Toledo en las elecciones de Abril 2019?*

Es decir, en cierto modo de lo estamos poniendo algo más fácil al RAG: primero con el reranking, y luego expresando la pregunta para que se parezca al formato que hemos utilizado para los metadata, y por lo tanto también para crear el índice FAISS. La respuesta del modelo fue bastante correcta.

Veremos que, cuando comprobemos la eficacia de RAG mediante RAGAS, es obvio que no contaremos con estas ventajas.

Evaluación RAGAS

Algo que desde luego no habíamos hecho en la práctica de LLM Engineering fue el evaluar el RAG, lo cual sí que es razonable llevar a cabo en el PFB. Hemos elegido la evaluación RAGAS, que tiende a considerarse como el estándar en la industria. Todo el proceso lo hemos incluido en el notebook '**Evaluacion_RAGAS.ipynb'**.

No estuvo el proceso de evaluación exento de dificultades, comenzando por el hecho de que el fichero de metadata.pkl tenía como formato subyacente una serie de pandas. Por otro lado, la instalación de Unsloth nunca es fácil tampoco, junto con la propia librería de ragas. Pese a todo, pudimos generar el vector_db a partir de FAISS.

En principio hemos tomado una serie de 10 preguntas/respuestas del dataset de text. Pensábamos que esta sería una prueba para luego proceder a una evaluación definitiva con 40. Con todo, la tardanza en ejecución de las 10, cerca de una hora, y la claridad de los resultados obtenidos, que mostramos más adelante, hemos pensado que lo hacen innecesario.

Como Ground Truth hemos tomado la respuesta presente en las preguntas/respuestas del dataset de text. Como Judge, inicialmente pensamos utilizar el propio modelo Gemma2, pero tuvimos problemas con la RAM de Colab, incluso aunque utilizásemos la GPU A100. Finalmente nos decidimos por usar el modelo v1 de DeepSeek, que pensamos que ha sido más prudente, porque no parece muy correcto que Gemma2 fuese juez y parte.

No podemos olvidar que, entre las 10 preguntas, existen muestras de los tres niveles. Con el nivel 3, el de las respuestas complicadas y largas, veremos que tuvimos problemas, pues el modelo v1 se hizo, interpretando lo que me comentó Gemini, demasiado quisquilloso.

Vamos con los resultados, que están presentes en el fichero '**resultados_evaluación_ragas_tfm_10.csv'**, que se encuentra en el directorio Dataset del repositorio de GitHub. Una tabla sencilla extraída del notebook es esta:

nivel	faithfulness	answer_correctness
nivel1	0.875162	0.372727
nivel2	0.894444	0.447622
nivel3	0.894737	NaN

Vemos que en cuestión de *faithfulness* es muy alto, pero que el *answer_correcteness* no lo es tanto. Por desgracia, RAGAS parece que no fue capaz de obtener respuestas para esta última métrica y, de hecho, una de las fuentes de ello es que tuvimos este error en la ejecución, que no fuimos capaces de reparar, incluso con la ayuda de Gemini:

<exception>

The output is incomplete due to a max_tokens length limit.

</exception>

De todas formas, le pasamos el fichero de resultados a Gemini para que lo interpretase, y nos devolvió esta tabla

Métrica	Estado	Interpretación
Faithfulness	**Muy Alta (en Nivel 1 y 2)**	Tu modelo es muy fiel. Si el dato está en el contexto, no se lo inventa. Esto es crítico para un TFM de datos electorales.
Context Recall	**Variable**	Aquí es donde veo más margen de mejora. Si el context_recall es bajo en algunas filas, significa que **FAISS no trajo la sección correcta**, por lo que el modelo, aunque quiera, no puede responder bien.
Answer Relevancy	**Alta**	El modelo entiende bien la intención del usuario y responde a lo que se le pregunta, sin irse por las ramas (excepto en el Nivel 3).

Es decir, que tenemos problemas con la selección de la sección correcta, es decir, de la fila correcta, que es uno de los desafíos que vimos en el diseño del RAG. En la evaluación RAGAS, no hemos 'adaptado' la pregunta, ni hemos utilizado un reranker, esas ventajas con las que probamos el modelo con el RAG.

Los comentarios adicionales de Gemini, que aclaran el error que vimos antes, pensamos que son interesantes:

1. El misterio de los NaN en Nivel 3

*Como sospechábamos por los errores de max_tokens, las preguntas de **Nivel 3** son las culpables.*

- ***Por qué ocurre:*** *El answer_correctness es una métrica que combina una comparación semántica con una descomposición de afirmaciones. Al ser respuestas tan largas y ricas en datos (como la que vimos de Móstoles y Madrid), el evaluador DeepSeek se queda sin espacio para cerrar el JSON. RAGAS, al recibir un JSON incompleto, no puede extraer la nota y pone NaN.*

- ***Opinión:*** *No significa que las respuestas sean malas, sino que **superan la capacidad de análisis actual del evaluador** con la configuración que usamos. Para la memoria, puedes reportar que el Nivel 3 muestra una complejidad que requiere una evaluación humana o un evaluador con una ventana de contexto mayor.*

El punto 2 es la tabla que hemos visto, y luego añade:

3. Fortalezas y Debilidades de tu RAG

- ***Fortaleza:*** *El modelo* ***Gemma-2 finetuneado*** *maneja excepcionalmente bien el formato de los datos electorales. No "alucina" con los números si el contexto es claro.*

- ***Debilidad (El "Talón de Aquiles"):*** *El sistema de recuperación (Retrieval). En algunos casos del CSV, veo que el contexto recuperado habla de municipios o años distintos a la pregunta. Esto penaliza todas las métricas.*

4. ¿Cómo poner esto en tu TFM? Mi recomendación:

Para la memoria, yo presentaría los datos agrupados. No digas "falló el nivel 3", di:

"Se observa una correlación inversa entre la longitud de la respuesta y la tasa de éxito del evaluador automático. Mientras que los niveles 1 y 2 muestran métricas de fidelidad superiores al 0.8, el nivel 3 (análisis complejo) presenta desafíos de tokenización en la fase de evaluación, sugiriendo la necesidad de métricas específicas para textos de larga extensión."

Pues ahí queda dicho. Se ve que el buen resultado del validation loss en el finetuning se refleja en un faithfulness muy alto.

Con todo, tenemos que estar pendientes de la selección de filas por parte del FAISS, que mejoraremos cuando implementemos la app mediante:

- Ampliando el número de filas seleccionadas con FAISS, y seleccionando de ellas las que coincidan con el municipio y la elección, que habrán sido seleccionadas anteriormente por el usuario.

- Uso del reranker.

- Añadir el nombre del municipio y la elección explícitamente al texto de la pregunta que haga el usuario.

Lo cierto es que, en conjunto, el RAGAS ha dado unos resultados mejores que lo que nos temíamos, pero además bastante útiles para intentar, al menos, optimizar la performance del modelo en la aplicación.

Despliegue de la aplicación en Google Cloud

Debemos acabar el PFB con un despliegue de una aplicación de utilice el modelo. Lo cierto es que se trataría de cargar el modelo en una plataforma, en este caso Google Cloud, mediante la cual el usuario pueda pedir un informe breve y rápido, o hacer preguntas, y que, de paso, incorpore alguna gráfica que aporte información.

La sensación que nos queda es que hemos podido elegir un camino demasiado complicado, y también caro, en el despliegue en sí.

El notebook donde ensayamos el código de la aplicación es el **'App_prueba_GitHub.ipynb'**. Este código no es el definitivo que se encuentra en Google Cloud, por supuesto, ya que hay que cambiar la ruta de carga del fichero .faiss, así como adaptar la carga de librerías a las que están presentes en la VM de Google Cloud. En Colab hemos ido probando la app mediante ngrok. El código definitivo de la app, el fichero **app-2.py**, se encuentra en el directorio App del repo de GitHub. No hemos incluido los acompañantes habituales, como el requirement.txt, que inicialmente sí subimos a la VM, pero dieron tantos problemas que entre Gemini y el que escribe tuvimos que ir paso a paso.

Esencialmente, la aplicación consiste en lo siguiente:

- Carga de ficheros .faiss, por un lado, y del modelo y dataset a través de Hugging Face
- Selecciona un municipio y una elección a través de un menú jerárquico.
- Gráficas con la librería Plotly de las secciones del municipio seleccionado y participación en función de la renta personal.
- Conexión con el modelo LLM aportando una pregunta genérica 'Analiza brevemente los datos del municipio {mun_selected} en las elecciones {elec_selected}', o por una pregunta introducida por el usuario. El modelo contesta brevemente, añadiendo una tabla con los datos relevantes de las secciones seleccionadas por el RAG.
- La aplicación incorpora memoria, haciendo posible que el modelo compare los datos de más de un municipio
- Hay un faldón inferior desplegable que muestra las filas elegidas por el RAG
- Hemos incorporado un sistema de trazabilidad con LangSmith en el proceso de selección de filas mediante FAISS y el reranker.

El prompt pasado al LLM se compone inicialmente por la instrucción:

> *### Instrucción:Eres un analista de datos experto. Utilizando exclusivamente el contexto actual y el historial proporcionado, responde a la pregunta del usuario.*
>
> *IMPORTANTE: No expliques cómo se hace el análisis, REALIZA el análisis cuantitativo directamente usando los números del contexto.*
>
> *Si hay varias secciones, compáralas y extrae una conclusión basada en las cifras de participación y renta.*

Analiza los datos de las secciones de {mun_selected} en la elección {elec_selected}. COMPARA los datos del municipio presente en el contexto actual, {mun_selected}, con los datos de los municipios presentes en el historial proporcionado.

Presenta primero una tabla de Markdown con los datos (Sección, Participación, Renta) y termina con una conclusión de unas 200 palabras.

A continuación de añaden el resto de elementos del prompt:

Historial proporcionado:

{historial_texto}

Pregunta:

{pregunta}

Contexto actual:

{contexto_rag}

Respuesta: Basado en el análisis de los datos electorales,"""

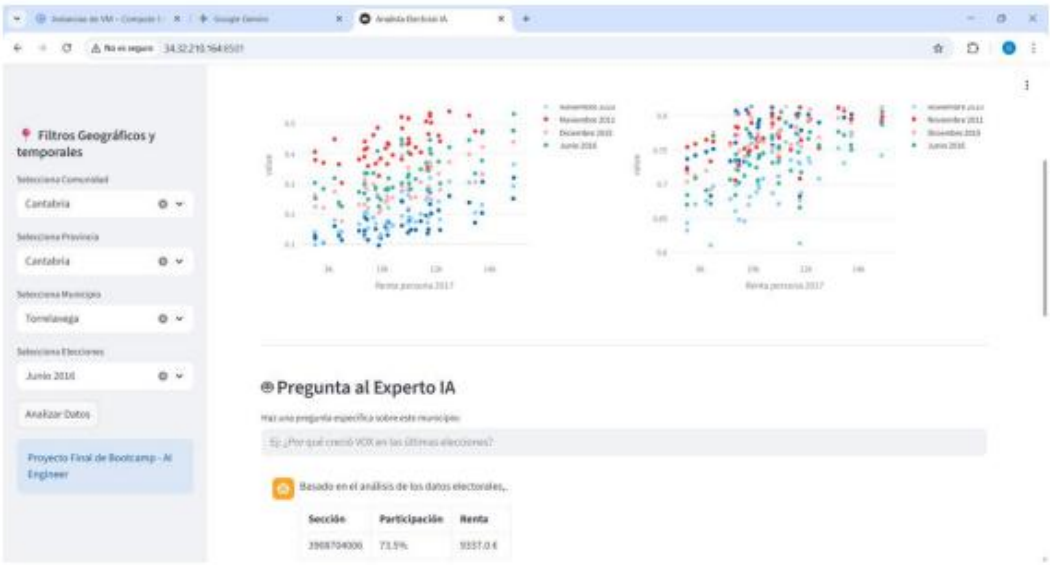
En cuanto al despliegue en sí en Google Cloud, lo cierto es que tuvimos un gran problema. Gemini me convenció de implementar una máquina virtual, en vez de usar Docker, debido a los problemas que podía haber con la conexión de los drivers de la GPU necesaria, que fue lo que al final nos pasó con la máquina virtual.

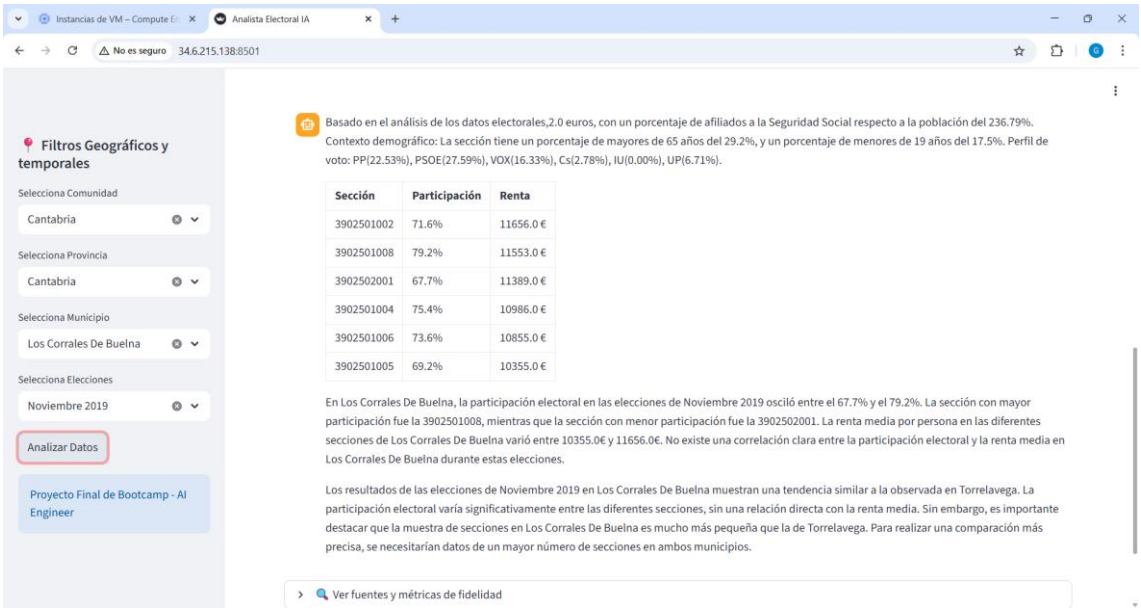
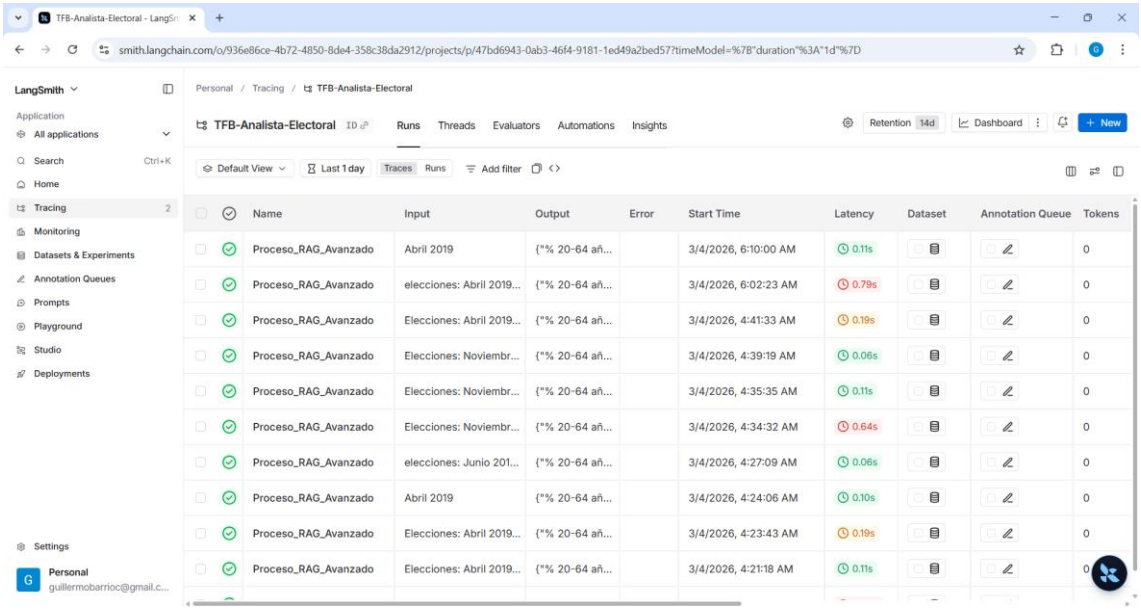
La VM que hemos levantado contiene una GPU de 15 GB, con una NVIDIA T4, con 4 núcleos de 30GB y un disco duro de 80 GB. Está previsto que se apague definitivamente el próximo 7 de abril, aunque por las tardes todos los días no siempre es posible conectarla por falta de capacidad en la región europe-west4a.

Por otro lado, fue una lucha enorme conectar internamente la propia VM con la GPU, lo cual requirió bajar los drivers desde la web de Nvidia. Además, una vez que subimos los ficheros necesarios, aparecieron problemas con la librería que Unsloth. Finalmente, fue posible desplegar la aplicación, y gravé un vídeo como demo de ella, que se ha subido a YouTube, y cuyo link es el siguiente:

https://www.youtube.com/watch?v=v0s5F2np7PM

Hemos aportado un par de capturas de pantalla de la app en funcionamiento. La primera de ellas muestra el aspecto superior de ella, con las gráficas de Plotly, en el caso del municipio de Torrelavega (Cantabria). En el segundo caso se puede observar la respuesta del modelo a una segunda selección de municipio, en este caso Los Corrales de Buelna, una localidad cerca de Torrelavega, y cómo el modelo es capaz de relacionar ambos municipios. Hay que añadir, y en el vídeo se muestra, que en un faldón desplegable se pueden consultar las filas del dataset seleccionadas por el sistema de filtrado mediante Faiss y el reranker.


Filtros Geográficos y temporales
Selecciona Comunidad
Cantabria
Selecciona Provincia
Cantabria
Selecciona Municipio
Torrelavega
Selecciona Elecciones
Junio 2016
Analizar Datos
Proyecto Final de Bootcamp - AI Engineer
Pregunta al Experto IA
Basado en el análisis de los datos electorales,.
Sección
Participación
Renta


Instancias de VM – Compute E
Analista Electoral IA
No es seguro
34.6.215.138:8501
Filtros Geográficos y temporales
Selecciona Comunidad
Cantabria
Selecciona Provincia
Cantabria
Selecciona Municipio
Los Corrales De Buelna
Selecciona Elecciones
Noviembre 2019
Analizar Datos
Proyecto Final de Bootcamp - AI Engineer
Basado en el análisis de los datos electorales,2.0 euros, con un porcentaje de afiliados a la Seguridad Social respecto a la población del 236.79%. Contexto demográfico: La sección tiene un porcentaje de mayores de 65 años del 29.2%, y un porcentaje de menores de 19 años del 17.5%. Perfil de voto: PP(22.53%), PSOE(27.59%), VOX(16.33%), Cs(2.78%), IU(0.00%), UP(6.71%).
Sección | Participación | Renta
3902501002 | 71.6% | 11656.0 €
3902501008 | 79.2% | 11553.0 €
3902502001 | 67.7% | 11389.0 €
3902501004 | 75.4% | 10986.0 €
3902501006 | 73.6% | 10855.0 €
3902501005 | 69.2% | 10355.0 €
En Los Corrales De Buelna, la participación electoral en las elecciones de Noviembre 2019 osciló entre el 67.7% y el 79.2%. La sección con mayor participación fue la 3902501008, mientras que la sección con menor participación fue la 3902502001. La renta media por persona en las diferentes secciones de Los Corrales De Buelna varió entre 10355.0€ y 11656.0€. No existe una correlación clara entre la participación electoral y la renta media en Los Corrales De Buelna durante estas elecciones.
Los resultados de las elecciones de Noviembre 2019 en Los Corrales De Buelna muestran una tendencia similar a la observada en Torrelavega. La participación electoral varía significativamente entre las diferentes secciones, sin una relación directa con la renta media. Sin embargo, es importante destacar que la muestra de secciones en Los Corrales De Buelna es mucho más pequeña que la de Torrelavega. Para realizar una comparación más precisa, se necesitarían datos de un mayor número de secciones en ambos municipios.
Ver fuentes y métricas de fidelidad

Finalmente, mostramos una captura de pantalla de la trazabilidad de LangSmith, correspondiente precisamente a las búsquedas realizadas durante la grabación del vídeo. Reconocemos que, como tal, no nos ha sido muy útil, pero pensamos que es importante al menos el haber sido capaz de ponerlo en marcha.